

Entwicklerhandbuch für die Umsetzung des Brettspieles 'Go'

Harry Flohr (6811837)
und
Jan Ole Wellnitz (6796226)

17. September 2017

1 Module der Software

1.1 Bildverarbeitung (go_bv.rkt)

Das Modul Bildverarbeitung ist für das Auslesen der Spielfeldbelegung eines Bildes zuständig. Ein Bild wird in das Modul geladen und dann verarbeitet. Es wird zuerst das Spielbrett aus dem Bild extrahiert und in ein Schwarz-Weiß-Bild gewandelt. Die Graustufen stellen die Anteile der blauen Farbe in dem originalen Bild dar. Das Bild wird anschließend noch verwaschen, um die Spielfeldlinien und Reflexionen leichter von den Spielsteinen unterscheiden zu können.

In diesem Bild wird die Koordinaten des ersten Spielfeldes (oben links) gesucht. An dieser Koordinate wird geschaut, ob die Schwellenwerte für einen weißen oder schwarzen Stein getroffen wird. Diese Überprüfung wird an insgesamt neun Stellen pro Koordinate vorgenommen. Es wird quadratisch um die Koordinate kontrolliert (siehe Abb. 1.1).

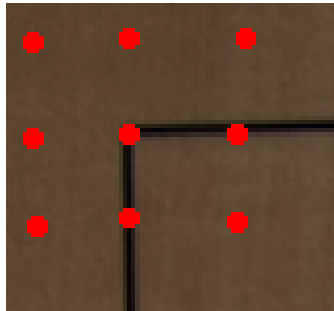


Abbildung 1.1: Suchemuster an Koordinate

Dieser Prozess wird an jeder möglichen Koordinate für einen Stein durchgeführt und so eine Spielfeldbelegung, wie in 4.1 beschreiben, erstellt. Die Koordinaten werden anhand zweier Schrittweiten (in x - und y-Richtung) ermittelt. Ist das ganze Spielfeld kontrolliert, hat das Modul die vollständige Spielfeldbelegung des Bildes projiziert.

1.2 Spielbrett zeichnen (go_draw_board.rkt)

Das Modul `go_draw_board.rkt` ist für die Anzeige des gesamten User-Interfaces zuständig.

Es zeichnet zum einen aus einer gültigen Spielfeldbelegung (4.1) das belegte Spielbrett. Zuerst wird das Grundbrett gezeichnet. Anschließend werden die Listen des Spielstandes ausgelesen und die benötigten Steine auf das Spielfeld gezeichnet. Die Leserichtung der Listen erfolgt von oben links nach unten rechts.

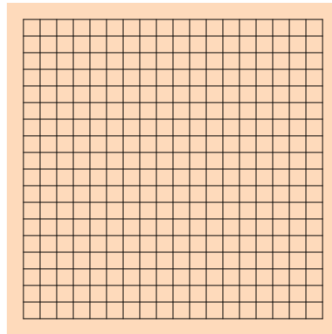


Abbildung 1.2: Leeres Spielfeld

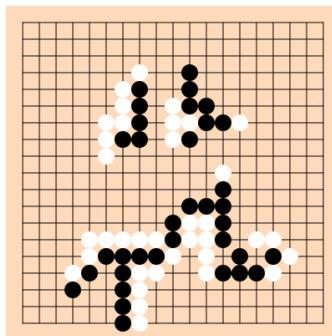


Abbildung 1.3: Belegtes Spielfeld

Zudem zeichnet das Modul beim Start eines Spieles die benötigten Auswahloptionen, die der Benutzer vor dem Starten eingeben muss. Das kann zum einen über das Darstellen einer Auswahlfläche (Abb. 1.4) geschehen, auf die der Benutzer klicken muss. Zum anderen aber auch eine Aufforderung zur Eingabe von Zahlen über die Tastatur (Abb. 1.5).

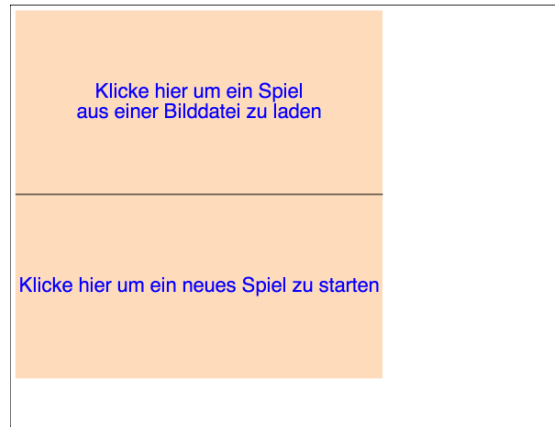


Abbildung 1.4: Auswahl über Optionsfelder

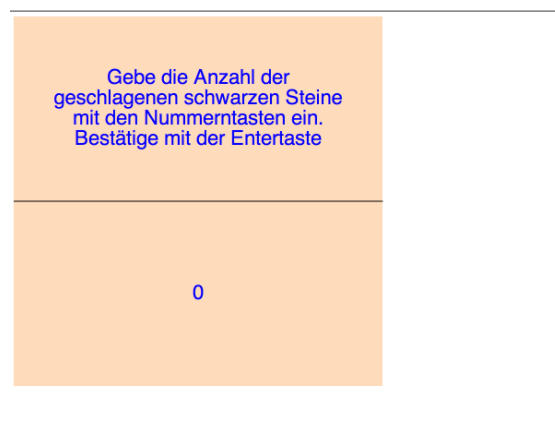


Abbildung 1.5: Zahleneingabe

Nach Eingabe aller Parameter erstellt die Komponente das User-Interface für das Spielen selbst. Dazu gehören das Spielfeld, Spielstatus des Spielers, die erzielten Punkte und Buttons (für Passen oder ein neues Spiel starten) (Abb. 1.6).

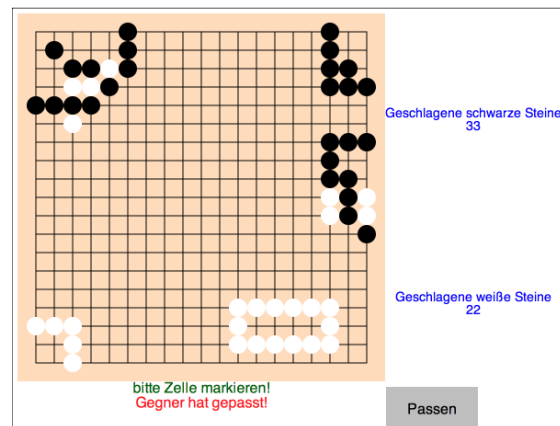


Abbildung 1.6: Das Spielfeld während des Spiels

Alle Darstellung des User-Interfaces können über die Schnittstellen des Moduls angezeigt werden. Soll eine z. B. Eingabeoption erstellt werden, muss dazu eine neue Funktion erstellt werden und als Schnittstelle bereit gestellt werden. Um bestehende Darstellungen zu ändern, müssen die entsprechenden Funktionen in diesem Modul angepasst werden.

1.3 Spielclient (go_client.rkt)

Der Spielclient steuert die Kommunikation des Spielers mit dem Server. Mit dem Spielclient kann auf zwei verschiedene Arten ein Spiel gestartet werden. Es kann ein komplett neues Spiel gestartet werden und es kann ein Spielstand mit einem Bild-Spielstand gestartet werden. Bei dem Start aus einem Bild-Spielstand fragt der Client ab, wie viele Steine bereits geschlagen wurden, welche Farbe der Spieler hat, ob ein Spieler gepasst hat und welcher Spieler am Zug ist. Diese Informationen werden dann zu dem Server geschickt und dieser initialisiert den ersten Spielstand (4.2). Die Funktionen zum zeichnen der Auswahloptionen und Eingaben stammen aus dem `go_draw_board.rkt` Modul.

Ebenfalls ist der Spielclient für die Erstellung des Interfaces zuständig. Über dieses zeigt er den Spielern während eines Spiels den aktuellen Spielstand an, ob sie an der Reihe sind, die Anzahl der geschlagenen Steine, ob der Gegner gepasst hat, sie einen ungültigen Zug vorgenommen haben und wer gewonnen hat. Auch die Buttons zum Passen oder für das Starten eines neuen Spiels, werden angezeigt. Diese Anzeige wird mit Hilfe des `go_draw_board.rkt` Modul erstellt.

Die Hauptaufgabe des Clients ist aber die Entgegennahme von Benutzereingaben. Der Client nimmt Click-Events und Tastatureingaben entgegen, leitet sie zum Server weiter, empfängt das Ergebnis und stellt das Ergebnis dar. So kann der Benutzer über den Client ein Spiel starten, die Spiel-Parameter eingeben, das Spiel bis zum Ende durchführen und anschließend ein neues Spiel starten.

1.4 Spielserver (go_server.rkt)

Der Spielserver leitet das Spiel. Zu Beginn erhält der Server die Information, ob er ein neues Spiel starten oder ein Spiel aus einem Bild laden soll. Der Server lädt das Bild und ermittelt den Spielstand. Der Server schickt dann ggf. eine Abfrage für die weiteren benötigten Informationen an den Client. Diese sind bei einem neuen Spiel die Vorgabe und bei einem Spielstand aus einem Bild die geschlagene Steine, der Spieler, der am Zug ist, und ob der vorherige Spieler gepasst hat. Anschließend sendet der Server den vollständigen Startzustand an die Clients.

Eine weitere Aufgabe während des gesamten Spieles ist die Verwaltung des Spielstandes (4.2) und welcher Spieler am Zug ist. Ebenfalls prüft der Server die übermittelten Spielzüge der Spieler auf ihre Gültigkeit und teilt den Spielern ggf. mit, dass der Spielzug nicht zulässig ist. Bei zulässigen Zügen passt der Server den Spielstand (4.2) an und übermittelt den vollständig erstellten Spielstand an die Clients.

Nach Beendigung eines Spieles, ermittelt der Server den Gewinner und teilt den Spielern (Clients) den Gewinner mit.

1.5 Spielserver (go_logic.rkt)

In dem Modul `go_logic.rkt` ist die Spiellogik implementiert. Diese kann in drei verschiedenen Aufgaben unterteilt werden.

1. Vorgaben machen:

Wenn bei dem Start eines neuen Spiels eine Vorgabe gewährt wird, kann der Spieler vor Spielbeginn die angegebene Anzahl Steine setzen.

2. Stein setzen:

Wenn ein Stein gesetzt wird, muss geprüft werden, dass dieser keinen Selbstmord begeht (Abb.). Das würde bedeuten, dass dieser Stein und ggf. seine anliegende Struktur durch das Setzen dieses Steines zerstört wird.

Dieser Zug ist nur erlaubt, wenn mit diesem Zug Steine des Gegners geschlagen werden und so der Stein und ggf. seine Struktur neue Freiheiten erhalten (Abb.).

Wenn ein Stein normal gesetzt wird, kann das Setzen zur Folge haben, dass durch diesen Zug eine Struktur des Gegners zerstört wird. Wenn dieser Fall eintritt müssen alle geschlagenen Steine von dem Spielfeld entfernt werden und die Punkte werden dem Spieler gut geschrieben.

3. Auswertung:

Nachdem beide Spieler nacheinander gepasst haben, ist das Spiel beendet. Dann startet die Auswertung. Bei der Auswertung müssen eroberte Gebiete gefunden werden. Das sind Flächen auf denen kein Stein liegt, die aber von einem Spieler eingeschlossen sind. Wenn ein Spieler eine freie Fläche erobert hat, erhält dieser die Anzahl an freien Positionen sowie die Anzahl der gegnerischen Steine in diesem Gebiet. Ein erobertes Gebiet ist in Abb. zu sehen. Nachdem alle Flächen überprüft sind, wird die Anzahl der erzielten Punkte auf die bereits erzielten Punkte aufaddiert.

Diese drei Hauptaufgaben können über Schnittstellen ausgeführt werden. Änderungen in der Spiellogik bzw. Erweiterungen sind demzufolge in diesem Modul zu erledigen.

2 Schnittstellen

2.1 Bildverarbeitung

1. (read-board path):

Das Modul bietet eine Funktion, um die Belegung des Spielfeldes auszulesen. An diese Funktion wird der Dateipfad des Bildes übergeben und anschließend liefert diese die Belegung des Spielfeldes, wie in 4.1 beschrieben, zurück.

2.2 Spielbrett zeichnen

1. (draw-board-with-score game_score):

Die Funktion zeichnet ein Spielbrett anhand eines in 4.1 definierten Spielfeldes. An diese Funktion wird die Belegung des Spielfeldes übergeben und als gezeichnetes Spielfeld mit Belegung zurückgeliefert.

2.3 Spielclient

1. (receive w m):

Der Spielclient erhält über die von dem Universe-Paket gelieferten Funktion seinen Spielstand (4.2). In dem Parameter w befindet sich der alte Weltzustand und in dem Parameter m der neue Weltzustand. Der neue Weltzustand m wird zuerst überprüft. Ist er gültig wird dieser angezeigt, ist er es nicht, wird der alte Weltzustand gewählt.

2. (make-package w m):

Der Spielclient kann mittels der Funktion (make-package w m) mit dem Server kommunizieren. So kann der Spielclient mit dem Parameter w seine aktuelle Welt senden und in dem Parameter m weitere Informationen verpacken. Zum Beispiel wird dort angegeben an welcher Position der Spieler seinen Stein gesetzt hat.

2.4 Spielserver

1. (make-mail to content):

Der Spielserver kann mit der Funktion make-mail Informationen an den Client senden. In dem Parameter to wird das Ziel des Aufrufes festgelegt. In dem content befindet sich der aktuelle Spielstand (4.2).

2. (handle-messages univ wrld m):

Der Server kann mit der Funktion handle-messages Daten von den Clients empfangen. In dem Parameter univ befindet sich das Universum selbst. In wrld befindet sich die von dem Client gesendete World und in dem Parameter m die Aktion, die der Client gerne ausführen möchte. Das kann z. B. das Setzen eines Steines sein. Dann befindet sich in m die Funktion 'set sowie die x- und y-Koordinaten der Position, an der der Stein gesetzt werden soll. Ebenfalls können sich in dem Parameter m die Informationen für den Spielstart befinden. Wie z. B. die Vorgabe oder die geschlagenen Steine.

3. (add-world univ wrld):

Mit der Funktion add-world kann sich ein Client bei dem Server anmelden. Dazu wird das Universum (univ) sowie die World (wrld) benötigt. Wenn zwei Clients angemeldet sind, werden weitere Clients abgewiesen.

3 Paketstruktur

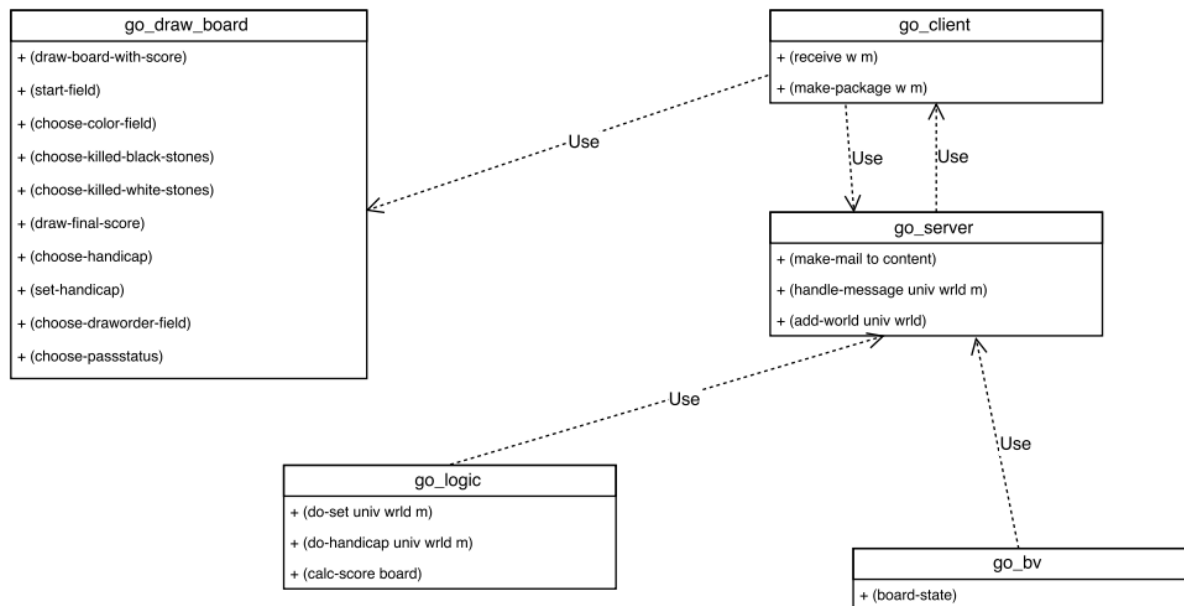


Abbildung 3.1: Paketstruktur

4 Strukturen

4.1 Spielfeld

Ein Spielfeld besteht aus 19x19 Felder, auf denen Steine positioniert werden können. Das Spielfeld wird in dem Programm als eine Liste mit 19 weiteren Listen dargestellt. Diese 19 Listen stehen für die y-Koordinaten des Spielfeldes. Jede dieser 19 Listen enthält 19 Positionen, um einen Wert einzutragen. Diese Unterlisten stehen für die x-Koordinaten.

Wenn an einer Position eine 0 steht, befindet sich kein Stein auf dem Spielfeld. Befindet sich dort eine 1, liegt an der Position ein weißer Stein. Bei einer -1, liegt an der Position ein schwarzer Stein.

Darstellung des Spielstandes in Abbildung 1.3:

```
'((0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 1 0 0 -1 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 1 -1 0 0 -1 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 1 -1 0 1 -1 -1 0 0 0 0 0 0 0)
(0 0 0 0 0 1 1 -1 0 1 1 -1 -1 1 0 0 0 0 0)
(0 0 0 0 0 1 -1 -1 0 1 -1 0 0 0 0 0 0 0 0)
(0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 -1 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 -1 -1 -1 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 -1 1 1 -1 0 0 0 0 0 0)
(0 0 0 0 1 1 1 1 1 -1 1 1 -1 0 1 1 0 0 0)
(0 0 0 0 1 -1 -1 -1 -1 1 0 1 0 -1 0 -1 1 0 0)
(0 0 0 1 -1 0 -1 1 1 0 0 1 -1 -1 -1 1 0 0 0)
(0 0 0 -1 0 0 -1 1 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 -1 1 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 -1 1 0 0 0 0 0 0 0 0 0 0 0))
```

4.2 World (Spielzustand)

Die World setzt sich aus mehreren Informationen zu einer Liste zusammen:

1. Spielfeld (siehe 4.1)
2. Weltzustand
Der Weltzustand gibt an, ob der Client ziehen darf, warten muss oder das Spiel beendet wurde (Zustände: wait, play, lost, won).
3. Geschlagene Steine
Die bereits geschlagenen Steine werden als Pair '(schwarz . weiß) übergeben.
4. Gepasster Spielzug
Hat der vorherige Spieler in seinem Spielzug gepasst und der Spieler an der Reihe passt ebenfalls, ist das Spiel beendet und die Auswertung startet (Gepasst: true/false).

Als Beispiel eine mögliche World-Darstellung bei der der aktuelle Spieler am Zug ist, 5 schwarze und 8 weiße Steine geschlagen wurden und der vorherige Spieler nicht gepasst hat. Der Spielstand ist zur Übersicht vereinfacht:

```
'(play '(spielfeld) '(5 . 8) false)
```

5 ToDo's

5.1 Dateiverarbeitung (`go_datei.rkt`)

Dieses Modul lädt aus einer Spielstand-Datei den aktuellen Spielstand und gibt diesen als World (4.2) zurück. Ebenfalls kann dieses Modul einen Spielstand in die Datei schreiben. Die Datei enthält folgende Informationen:

1. Belegung des Spielfeldes (4.1)
2. Anzahl geschlagener Steine (schwarz . weiß)
3. Der Spieler, der am Zug ist
4. Passstatus des vorherigen Spielers